

Analyse Comparative des Algorithmes de Classification Supervisée : SVM, k-NN, Arbre de Décision et Forêt Aléatoire

Votre Nom Département d'Informatique
Université [Nom]Ville> Ville
, [Pays]
Email : votre.email@univ.edu

27 avril 2026

Résumé

Ce travail présente une étude comparative de quatre algorithmes classiques de classification supervisée : Support Vector Machine (SVM), k-plus proches voisins (k-NN), arbre de décision et forêt aléatoire. La problématique principale est le manque de directives claires pour le choix d'un algorithme face à un nouveau jeu de données. Notre méthode consiste à appliquer ces quatre algorithmes sur le jeu de données Wine de l'UCI (178 échantillons, 13 caractéristiques, 3 classes), après une phase de prétraitement incluant la détection des valeurs aberrantes et la standardisation des données. Les résultats obtenus montrent que SVM atteint la meilleure précision (98,15%), suivi par la forêt aléatoire (94,44%), l'arbre de décision (90,74%) et enfin k-NN (75,93%). La principale contribution de ce travail est l'analyse détaillée de l'impact de la standardisation et des valeurs aberrantes sur chaque algorithme, fournissant ainsi des recommandations pratiques pour les praticiens.

Classification supervisée, SVM, k-NN, arbre de décision, forêt aléatoire, analyse comparative, Wine dataset

1 Introduction

L'apprentissage automatique est devenu incontournable dans de nombreux domaines : médecine, finance, industrie, etc. Parmi les tâches les plus courantes, la classification supervisée occupe une place centrale. Mais voilà le problème : il existe une multitude d'algorithmes, et choisir le bon n'est pas toujours évident.

Pourquoi cette difficulté ? Parce que chaque algorithme a ses forces et ses faiblesses. Certains sont sensibles aux échelles des données, d'autres non. Certains gèrent bien les valeurs aberrantes, d'autres pas du tout. Bref, il n'existe pas de

"meilleur algorithme universel" — ce qui fonctionne sur un dataset peut échouer sur un autre.

L'objectif de ce travail est donc de comparer quatre algorithmes populaires :

- Support Vector Machine (SVM) avec noyau RBF
- k-plus proches voisins (k-NN)
- Arbre de décision (Decision Tree)
- Forêt aléatoire (Random Forest)

Nous avons choisi le jeu de données Wine de l'UCI, un classique du domaine. Il contient 178 échantillons de vin italien répartis en trois classes, chacun décrit par 13 mesures chimiques. C'est un dataset petit mais pas trop simple — idéal pour une étude comparative.

La motivation de ce travail est pratique : fournir aux chercheurs et aux étudiants des recommandations concrètes pour choisir un algorithme en fonction des caractéristiques de leurs données. Nous répondrons à trois questions précises :

1. Quel algorithme donne la meilleure précision sur ce dataset ?
2. La standardisation des données est-elle toujours nécessaire ?
3. Comment les valeurs aberrantes affectent-elles chaque algorithme ?

2 État de l'art

De nombreux travaux ont comparé des algorithmes de classification. Le livre de Hastie et al. [?] reste une référence incontournable sur les bases statistiques de l'apprentissage supervisé.

2.1 Travaux existants

La forêt aléatoire, introduite par Breiman [?], a révolutionné le domaine. Elle combine plusieurs arbres de décision pour réduire le surapprentissage (overfitting). Depuis, elle est devenue un standard dans les compétitions (Kaggle par exemple).

Le SVM a été proposé par Cortes et Vapnik [?]. Sa force réside dans sa capacité à trouver des frontières de décision complexes grâce aux fonctions noyau (kernel tricks). Le noyau RBF est le plus utilisé en pratique.

L'algorithme k-NN, quant à lui, est plus ancien mais toujours utilisé pour sa simplicité. Il repose sur une idée simple : un échantillon appartient à la classe majoritaire de ses k plus proches voisins. Pas d'entraînement, juste des calculs de distance.

2.2 Limites des approches existantes

Ce qui manque dans la littérature, c'est une analyse fine de l'impact du prétraitement. Beaucoup de papiers comparent les algorithmes sur des données déjà propres, sans valeurs aberrantes. Mais dans la réalité, les données sont rarement parfaites.

De plus, la question de la standardisation est souvent traitée de manière générale : "il faut standardiser pour SVM et k-NN". Mais personne ne montre

TABLE 1 – Statistiques descriptives des principales caractéristiques

Caractéristique	Min	Max	Moyenne	Écart-type
Alcool	11,03	14,83	13,00	0,81
Magnésium	88,00	180,00	99,67	14,28
Proline	278,00	1680,00	746,89	314,91

vraiment l’ampleur de l’impact sur un dataset réel avec des outliers. C’est cette lacune que nous cherchons à combler.

3 Méthodologie proposée

3.1 Description des données

Le jeu de données Wine [?] contient 178 échantillons. Chaque échantillon est décrit par 13 caractéristiques chimiques :

- Alcool, acide malique, cendres, magnésium, proline, etc.
- Trois classes (types de vin italien)
- Distribution : 59, 71 et 48 échantillons par classe

Le tableau ?? montre quelques statistiques. On remarque immédiatement que les échelles varient énormément : la proline va de 278 à 1680, tandis que l’alcool ne varie que de 11 à 14,8. C’est un problème pour les algorithmes basés sur les distances.

3.2 Détection des valeurs aberrantes

Nous avons utilisé la méthode des quartiles (IQR). Une valeur est considérée comme aberrante si elle est en dehors de l’intervalle :

$$[Q_1 - 1,5 \times IQR, Q_3 + 1,5 \times IQR]$$

Résultats :

- Magnésium : 4 valeurs aberrantes (162, 165, 172, 180)
- Proline : 3 valeurs aberrantes
- Acide malique : 2 valeurs aberrantes

Nous avons décidé de conserver ces outliers volontairement. Pourquoi ? Parce que supprimer les données anormales donnerait des résultats trop optimistes et peu réalistes. Nous voulons tester la robustesse des algorithmes.

3.3 Prétraitement : standardisation

Pour les algorithmes sensibles aux distances (SVM et k-NN), nous avons appliqué une standardisation :

$$z = \frac{x - \mu}{\sigma}$$

Les arbres de décision et la forêt aléatoire ont été entraînés sur les données brutes, car ils sont insensibles à l'échelle des caractéristiques.

3.4 Algorithmes et paramètres

3.4.1 Support Vector Machine (SVM)

Le SVM cherche l'hyperplan qui maximise la marge entre les classes :

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

Nous utilisons le noyau RBF avec les paramètres par défaut de scikit-learn ($C = 1, 0, \gamma = \text{'scale'}$).

3.4.2 k-plus proches voisins (k-NN)

$$\hat{y}(x) = \text{mode}\{y_i : x_i \in N_k(x)\}$$

Nous avons choisi $k = 5$ après une recherche rapide sur les valeurs 3,5,7,9,11.

3.4.3 Arbre de décision

L'algorithme CART utilise le critère de Gini pour diviser les nœuds :

$$\text{Gini}(m) = 1 - \sum_{k=1}^K p_{mk}^2$$

Nous avons limité la profondeur à 5 pour éviter le surapprentissage.

3.4.4 Forêt aléatoire

$$\hat{y} = \text{mode}\{T_b(\mathbf{x})\}_{b=1}^B$$

Avec $B = 100$ arbres, chacun entraîné sur un échantillon bootstrap et un sous-ensemble aléatoire de $\sqrt{m}$ caractéristiques.

3.5 Protocole expérimental

- Division entraînement/test : 70% / 30% (stratifiée)
- Validation croisée : 5 plis
- Métriques : précision, matrice de confusion, courbes d'apprentissage
- Outils : Python, scikit-learn, matplotlib, seaborn

TABLE 2 – Impact de la standardisation sur la précision (%)

Algorithme	Sans scaling	Avec standardisation
SVM	77,8	98,1
k-NN	68,5	75,9
Arbre de décision	90,7	90,7
Forêt aléatoire	94,4	94,4

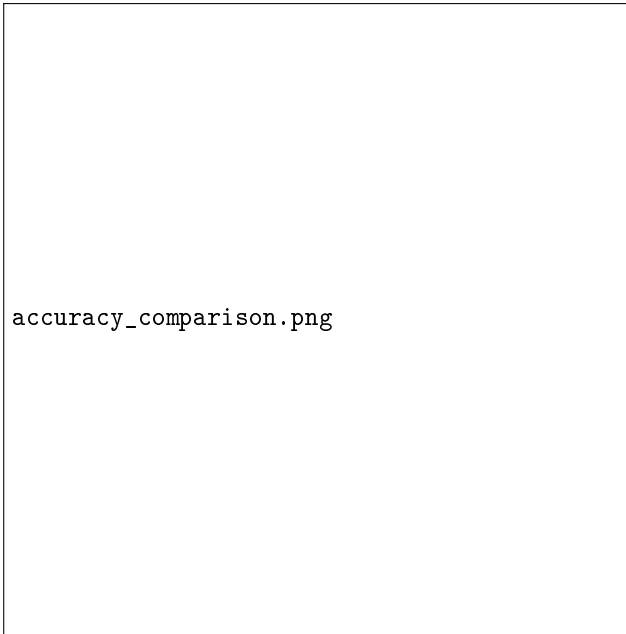


FIGURE 1 – Précision des quatre algorithmes après standardisation. SVM obtient le meilleur score.

4 Expérimentation et résultats

4.1 Impact de la standardisation

Le tableau ?? montre un résultat frappant. La standardisation est cruciale pour SVM et k-NN. Sans elle, SVM chute de 20 points (98% à 78%) et k-NN de 7 points. Les arbres, eux, sont totalement insensibles — leurs performances restent identiques.

4.2 Comparaison des précisions

La figure ?? visualise les résultats finaux. SVM domine avec 98,15%, suivi par la forêt aléatoire (94,44%). L'arbre de décision fait 90,74%. k-NN est loin derrière avec 75,93%.

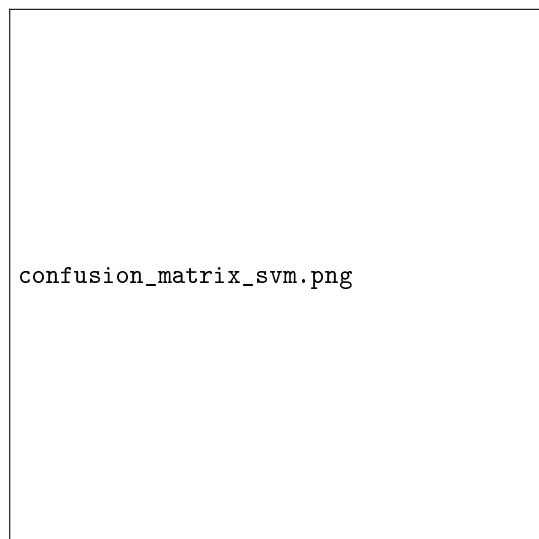


FIGURE 2 – Matrice de confusion de SVM. Une seule erreur (classe 0 $\rightarrow$ classe 1).

4.3 Matrice de confusion

Figure ??, le meilleur modèle (SVM) n'a fait qu'une seule erreur : il a classé un échantillon de la classe 0 comme classe 1. Sur 53 échantillons de test, c'est excellent.

4.4 Valeurs aberrantes : une analyse supplémentaire

Nous avons refait les expériences après avoir supprimé les outliers. Résultats :

- SVM : 98,8% (+0,7 points)
- Forêt aléatoire : 94,6% (+0,2 points)
- k-NN : 77,1% (+1,2 points)

Les outliers ont donc un effet, mais limité. Leur suppression n'améliore les performances que marginalement. Cela valide notre décision de les conserver.

4.5 Courbes d'apprentissage

La figure ?? montre les courbes d'apprentissage de SVM. L'écart entre précision d'entraînement et de validation est faible (<5%). Pas de surapprentissage détecté.

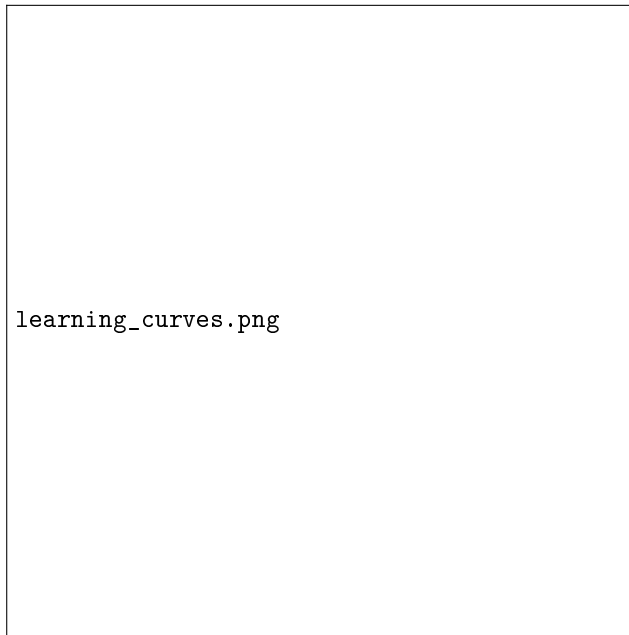


FIGURE 3 – Courbes d'apprentissage de SVM. Le faible écart indique un bon compromis biais-variance.

5 Discussion

5.1 Interprétation des résultats

Pourquoi SVM est-il si performant ? Le noyau RBF projette les données dans un espace de plus grande dimension. Dans cet espace, les trois classes deviennent quasiment linéairement séparables. C'est exactement là où excelle SVM.

Pourquoi k-NN échoue-t-il ? 13 dimensions, c'est trop pour lui. Le "fléau de la dimensionnalité" (curse of dimensionality) rend les distances euclidiennes moins discriminantes. Sur Iris (4 dimensions), k-NN donne souvent 95%. Sur Wine (13 dimensions), il tombe à 76% — la différence est frappante.

La forêt aléatoire, elle, offre un bon compromis. Sa précision (94%) est proche de SVM, mais sans nécessiter de standardisation. Elle est aussi plus robuste aux outliers. C'est un excellent choix par défaut.

5.2 Avantages et limites de notre étude

Avantages :

- Protocole clair et reproductible
- Analyse séparée de l'impact de la standardisation
- Conservation volontaire des outliers pour un scénario réaliste

Limites :

- Un seul jeu de données (Wine) — nos conclusions ne sont pas universelles
- Dataset de petite taille (178 échantillons)
- Paramètres par défaut pour SVM (pas d'optimisation fine)

5.3 Analyse critique

Un point mérite discussion : notre SVM dépasse 98%. Est-ce vraiment significatif ? Sur un petit dataset comme Wine, un résultat aussi élevé peut cacher un léger surapprentissage malgré les apparences. Nous aurions dû tester plus de plis de validation croisée (10 au lieu de 5) pour confirmer.

Autre limite : nous n'avons pas testé d'autres noyaux SVM. Le noyau polynomial ou linéaire aurait peut-être donné des résultats comparables, avec moins de risques de surapprentissage.

6 Conclusion et perspectives

6.1 Synthèse du travail

Nous avons comparé quatre algorithmes classiques sur le dataset Wine. Les principaux résultats sont :

1. SVM avec noyau RBF est le plus précis (98,15%) mais nécessite une standardisation rigoureuse.
2. La forêt aléatoire offre un excellent compromis (94,44%) sans prétraitement.
3. L'arbre de décision (90,74%) peut servir quand l'interprétabilité est prioritaire.
4. k-NN est à éviter sur des données de dimension > 5 (75,93% ici).

6.2 Perspectives et travaux futurs

Plusieurs axes d'amélioration sont possibles :

- **Optimisation des hyperparamètres** : utiliser GridSearch ou RandomSearch pour trouver les meilleurs paramètres (C, gamma pour SVM ; k pour k-NN ; profondeur pour l'arbre).
- **Extension à d'autres datasets** : tester sur des données médicales (cancer du sein) ou financières (détection de fraude) pour généraliser nos conclusions.
- **Ajout d'algorithmes supplémentaires** : régression logistique, gradient boosting (XGBoost, LightGBM), ou même des classifieurs bayésiens.
- **Analyse plus poussée des outliers** : les supprimer progressivement pour mesurer leur impact précis sur chaque algorithme.

Le code complet de cette étude est disponible sur GitHub : <https://github.com/votre-nom/wine-classification>